Round-trip closure on directed k-nearest-neighbour graphs: vote kernels, multiscale clustering, and structural pruning

Eraldo P. Marinho[1*], Caetano Mazzoni Ranieri[1] and Fabricio A. Breve[1]

[1*]Department of Statistics, Applied Mathematics and Computing, São Paulo State University (UNESP), Institute of Geosciences and Exact Sciences, Av. 24-A, 1515, Rio Claro, São Paulo, Brazil.

*Corresponding author(s). E-mail(s): pereira.marinho@unesp.br;
Contributing authors: cm.ranieri@unesp.br; fabricio.breve@unesp.br;

Abstract

We present RTKNNC, a graph-based clustering method that recursively combines forward KNN selection with reverse-neighbour retrieval on directed graphs. Weighted incoming links define popularity, which modulates a local RMS visibility threshold. Traversal over visible links stops at saturated closures, and singleton closures are rejected. An optional stage examines large closures for distinct internal populations without using reference labels. It fits diagonal Gaussian mixtures and rebuilds the cluster layer only when the proposed subdivision satisfies BIC, size, and visible-graph coherence criteria. We also compare uniform, inverse-distance, inverse-square, and sigmoid vote kernels under a common per-source vote budget. The historical inverse-square C and Python implementations produced identical partitions in 120 comparisons over eight datasets and $\boldsymbol{K = 2, \ldots, 16}$; the four-kernel sweep contained 840 observations. Across 210 dataset–scale configurations, the kernel changed the cluster count in 60 and the mean visible degree in all 210. At $\boldsymbol{K = 8}$, structural pruning changed the variable-density result from four clusters and ARI **0.7817** to five clusters and ARI **0.9627**. On the sparse bridge, it raised ARI from **0.8083** to **0.9853** at $\boldsymbol{K = 6}$ and from **0** to **0.9865** at $\boldsymbol{K = 10}$, while three correctly clustered controls remained at ARI **1**. These corrected cluster counts persisted under inverse-distance, inverse-square, and sigmoid voting without retuning the pruning rule. In an oracle-envelope comparison with seven external methods, pruned RTKNNC tied the perfect controls, approached the best variable-density result (**0.9627** versus **0.9639**), and obtained the highest sparse-bridge ARI (**0.9865** versus

0.9831). The experiments distinguish the effects of vote allocation, connectivity closure, and post-closure structural correction.

Keywords: k-nearest-neighbour graph, graph-based clustering, reverse nearest neighbours, round-trip closure, vote kernel, reverse-neighbour popularity, visibility, structural pruning, Gaussian mixture, multiscale clustering

1 Introduction

Nearest-neighbour relations are among the most fundamental structures for representing finite datasets embedded in metric spaces. They occur naturally in pattern classification, nonparametric density estimation, graph construction, manifold learning, outlier detection, and clustering [1–5]. In clustering, k-nearest-neighbour (KNN) relations have been explored through shared nearest neighbours, reciprocal nearest neighbours, mutual KNN graphs, reverse nearest-neighbour statistics, and density-based graph connectivity [6–11].

Beyond static point clouds, clustering also serves structured scientific data. For example, Marinho et al. [12] used it to identify dynamical regimes in time series derived from simulated orbits of Saturnian satellite clones.

In a directed KNN graph, each observation is a vertex, and an arc $x_i \to x_j$ is present whenever x_j is among the K nearest neighbours of x_i. RTKNNC retains this direction. Starting from a query point, it explores admissible outgoing neighbours; reaching x_j also exposes the points that selected x_j, namely

$$\mathcal{N}_K^-(x_j) = \{x_\ell \in X : x_j \in \mathcal{N}_K^+(x_\ell)\}. \tag{1}$$

Every reached point can repeat both traversals. Weighted incoming choices also define popularity, which modulates a pointwise RMS visibility threshold; only visible arcs participate in the expansion. Recursion stops at a fixed point, so a cluster candidate is a saturated closure of this locally filtered relation, rather than the result of a centroid, a prescribed shape, or a global distance threshold. Singleton closures are rejected.

This use of direction is the central idea: outgoing arcs encode whom a point selects, whereas incoming arcs encode by whom it is selected. RTKNNC uses the second view both to retrieve reverse neighbours and to regulate which local connections the closure may traverse. Repeating the construction independently for different values of K produces cluster layers that reveal persistence, absorption, splitting, and merging across neighbourhood scales. Because any connectivity closure can join structures through a small cut set, an optional post-closure stage accepts only strongly supported, graph-coherent subdivisions and rebuilds the final layer without modifying the KNN adjacency lists.

KNN graphs, shared-neighbour evidence, and reverse or reciprocal neighbours are established ingredients [6–8, 10, 11]; the contribution is their organisation into the filtered recursive closure above. Recent KNNG-CS work also derives representativeness

from weighted incoming KNN links [13], but uses it for greedy coreset selection rather than for visibility-controlled cluster formation.

1.1 Main contributions

The main contributions of this paper are as follows:

1. a formal, visibility-filtered round-trip closure on directed KNN graphs, combining outgoing selection, reverse-neighbour retrieval, popularity-modulated RMS visibility, saturation, and singleton rejection;
2. a source-budget-matched family of reverse-neighbour vote kernels, with exact recovery of the historical inverse-square implementation;
3. a K-indexed multiscale layer representation and an optional native structural-pruning rule based on BIC, minimum size, and visible-graph coherence; and
4. reproducible C–Python regression gates, controlled kernel and native sensitivity experiments, and comparison with seven external clustering methods.

2 Related work

2.1 Shared-neighbour and Jarvis–Patrick clustering

Jarvis and Patrick introduced a clustering criterion based on shared near neighbours [6]. Later shared-nearest-neighbour (SNN) clustering methods used common neighbour counts to improve robustness to clusters of different shapes, sizes, and densities [7]. RTKNNC is related to this family because reverse-neighbourhood intersections imply shared-neighbour evidence. Rather than using this evidence as a static similarity score, RTKNNC embeds it in a recursive graph-closure process.

2.2 Mutual, reciprocal, and reverse nearest-neighbour methods

Mutual KNN and reciprocal nearest-neighbour graphs have been used for clustering and outlier detection [8, 9, 14]. Reverse nearest-neighbour methods use incoming KNN links as density or representativeness signals [10]. RTKNNC uses reverse-neighbour sets both for backward traversal and to compute kernel-weighted popularity. Popularity then modulates the local visibility threshold for forward and backward transitions.

Recent work continues to develop neighbour relations as clustering primitives. MGNR constructs task-adapted multi-granularity relations [15]; weighted-KNN density peaks use a point's propensity to occur in other observations' KNN sets as density evidence [16]; and KNEG-CL evolves an unweighted directed KNN graph with K and uses in- and out-degree patterns for density-peak detection and subcluster merging [17]. These approaches pursue multi-granularity representation, density estimation, and degree-driven merging, respectively, whereas RTKNNC builds visibility-filtered recursive closures and studies their evolution as cluster layers.

KNNG-CS is particularly close in its definition of popularity [13]. It assigns every outgoing KNN edge a source-normalised softmax weight and defines node importance

as the sum of incoming weights. KNNG-CS computes this quantity once, greedily selects coreset representatives, and removes each selected point together with its reverse neighbours. RTKNNC instead uses distance-weighted incoming votes to modulate visibility during recursive forward–backward closure. KNNG-CS fixes each source's outgoing mass to one; RTKNNC retains the geometry-dependent historical budget $B_i = \sum_j 1/r_{ij}^2$. This common budget recovers the inverse-square operator exactly and permits comparison of alternative allocation shapes.

2.3 Density-based and spectral graph clustering

DBSCAN, OPTICS, and HDBSCAN define clusters through density connectivity and hierarchical density estimates [18–21]. Spectral clustering constructs a similarity graph and uses eigenvectors of the associated graph Laplacian to reveal cluster structure [5, 22]. The method proposed here is graph-based, but it is combinatorial and closure-driven: it does not require eigenvalue decomposition and it exposes a sequence of partitions as the neighbourhood size K varies. The comparison also includes classical agglomerative clustering [23], k-means [24, 25], and Gaussian mixtures, using the scikit-learn implementations [26].

3 Relational basis of the round-trip operator

The primitive idea can be stated for an arbitrary directed binary relation on a common finite set. Let

$$X = \{x_1, \ldots, x_N\} \tag{2}$$

be a finite set and let

$$T \subseteq X \times X \tag{3}$$

be a directed binary relation. Its symmetrised forward–backward, or round-trip, relation is

$$R_T = T \cup T^{-1} \subseteq X \times X. \tag{4}$$

For $x \in X$, the corresponding one-step round-trip neighbourhood is

$$R_T(x) = \{y \in X : (x, y) \in T\} \cup \{y \in X : (y, x) \in T\}. \tag{5}$$

Thus, one step retrieves both the objects selected directly by x and the objects that select x.

For a subset $S \subseteq X$, define

$$R_T(S) = \bigcup_{x \in S} R_T(x). \tag{6}$$

The cumulative round-trip closure starts from

$$C_T^0(S) = S, \tag{7}$$

and evolves according to

$$C_T^{t+1}(S) = C_T^t(S) \cup R_T(C_T^t(S)), \qquad t \geq 0. \tag{8}$$

The process saturates when

$$C_T^{s+1}(S) = C_T^s(S) \tag{9}$$

for the first time. The integer s is the saturation depth. Because X is finite and the sequence is monotone, saturation is guaranteed; the saturated set is the union of the weakly connected components of (X, T) that intersect S.

This relational view exactly describes the forward–backward mechanism used by the proposed graph operator. The algorithmic construction below instantiates T as the visibility-filtered edge relation of a directed KNN graph.

4 Directed KNN graphs and reverse neighbourhoods

Let

$$X = \{x_1, \ldots, x_N\} \subset \mathbb{R}^d \tag{10}$$

be a finite dataset with a metric $\delta : X \times X \to \mathbb{R}_{\geq 0}$. For each point x_i, let

$$\mathcal{N}_K^+(x_i) = \mathrm{KNN}_K(x_i) \tag{11}$$

denote the set of its K nearest proper neighbours. The directed KNN graph is

$$\mathcal{G}_K = (X, E_K), \tag{12}$$

where

$$(x_i, x_j) \in E_K \quad \Longleftrightarrow \quad x_j \in \mathcal{N}_K^+(x_i). \tag{13}$$

The reverse-neighbour set of x_j is

$$\mathcal{N}_K^-(x_j) = \mathrm{RNN}_K(x_j) = \{x_i \in X : x_j \in \mathcal{N}_K^+(x_i)\}. \tag{14}$$

The historical prototype also organised local popularity preferences into a leadership graph and then converted each of its weakly connected components into a rooted tree. Figure 2 records this provenance. These diagrams motivate the interpretation of reverse-neighbour popularity, but they are not an additional processing stage in the controlled vote-kernel benchmark reported below.

5 Vote-kernel popularity, visibility, and round-trip closure

The reference formulation adopts a popularity-modulated, visibility-filtered round-trip expansion. A point need not propagate through all of its K nearest-neighbour links: only links admitted by its local visibility threshold participate in the forward–backward closure. To reproduce the reference implementation exactly, linear and squared distances must be distinguished.

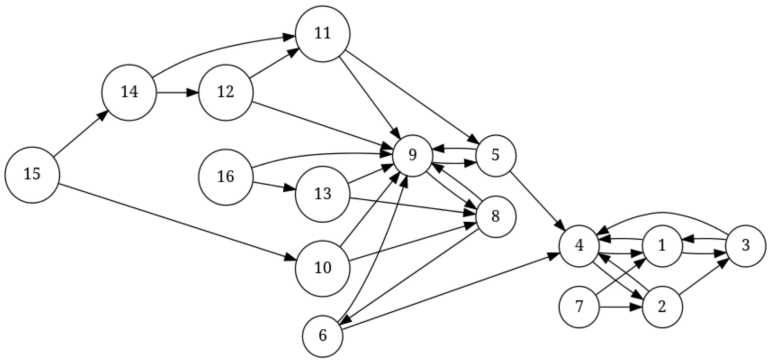


Fig. 1 Example of a directed KNN graph with $K = 2$, used to illustrate the proposed mechanism. Arrows represent directed nearest-neighbour choices. The graph encodes neighbourhood connectivity, but its abstract layout does not reproduce the spatial morphology of the original point distribution. Metric distances are stored as edge attributes and are not represented by the drawn edge lengths. The same graph supports reverse-neighbour retrieval and visibility-filtered round-trip expansion.

5.1 Distance representation and legacy radius

Let

$$r_{ij} = \delta(x_i, x_j), \qquad D_{ij} = r_{ij}^2. \tag{15}$$

The C implementation stores D_{ij}, rather than r_{ij}, in each KNN list. After the list is sorted, its stored radius is the largest retained squared distance,

$$R_K^{(2)}(x_i) = D_{i,(K)} = \max_{x_j \in \mathcal{N}_K^+(x_i)} D_{ij}. \tag{16}$$

The corresponding linear scale is

$$R_K(x_i) = \sqrt{R_K^{(2)}(x_i)} = r_{i,(K)}. \tag{17}$$

5.2 Vote kernels and reverse-neighbour popularity

For each source point x_i, define the dimensionless neighbour coordinate

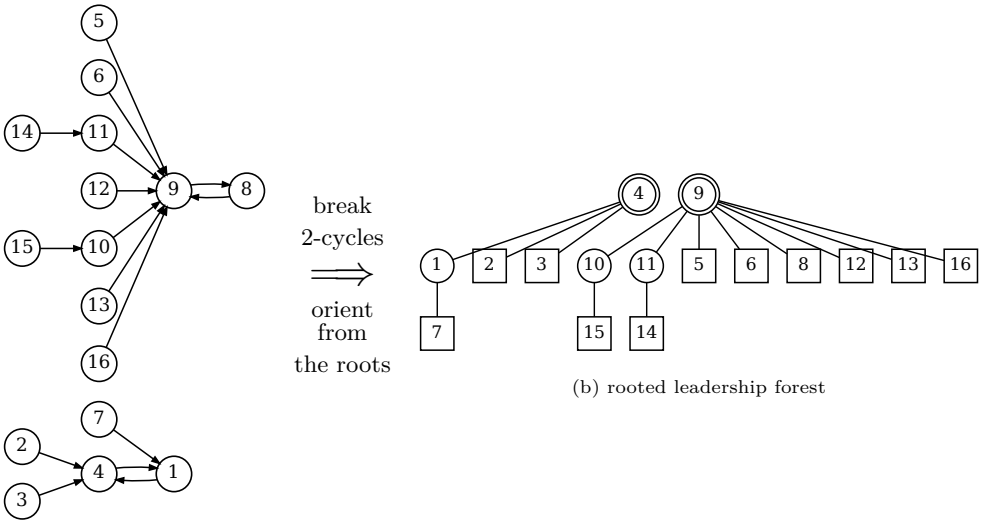
$$u_{ij} = \frac{r_{ij}}{R_K(x_i)}, \qquad x_j \in \mathcal{N}_K^+(x_i), \tag{18}$$

and compare the four allocation shapes

$$h(u) \in \left\{1, \frac{1}{u}, \frac{1}{u^2}, S_\alpha(u)\right\}, \tag{19}$$

where

$$S_\alpha(u) = \frac{1}{1 + \exp[\alpha(u-1)]}. \tag{20}$$



(a) directed leadership graph

(b) rooted leadership forest

Fig. 2 From the historical leadership graph to its rooted forest for the same 16-point KNN example. In (a), every vertex points to its locally selected leader; the graph has two weakly connected components, each containing a reciprocal leader pair, $1 \leftrightarrow 4$ and $8 \leftrightarrow 9$. Resolving each pair at its more popular vertex and displaying parent–child links away from roots 4 and 9 produces the two trees in (b). Vertex identifiers are preserved across panels, while nodes and edges are drawn uniformly in black so that colour carries no undocumented semantics. Double circles mark roots, single circles internal leaders, and squares leaves.

Here, $\alpha > 0$ controls the slope, and $R_K(x_i)$ supplies the local distance scale for every shape. Directly comparing the raw values 1, $1/r$, $1/r^2$, and $S_\alpha(r/R_i)$ would confound shape with units and arbitrary outgoing amplitude. We therefore retain the historical inverse-square vote mass of source i as the common budget

$$B_K(x_i) = \sum_{x_k \in \mathcal{N}_K^+(x_i)} \frac{1}{r_{ik}^2}, \tag{21}$$

and redistribute that budget according to the selected shape:

$$g_{i,h}(x_j) = B_K(x_i) \frac{h(u_{ij})}{\sum_{x_k \in \mathcal{N}_K^+(x_i)} h(u_{ik})}. \tag{22}$$

Thus $\sum_j g_{i,h}(x_j) = B_K(x_i)$ for every model. For a fixed allocation shape h, let $g_h = \{g_{i,h}\}_{i=1}^N$ denote the resulting vote model; when the selected shape is clear, we write $g = g_h$. The incoming popularity of x_j under that model is

$$P_{K,g}(x_j) = \sum_{x_i :\, x_j \in \mathcal{N}_K^+(x_i)} g_{i,h}(x_j). \tag{23}$$

Thus, $P_{K,g}(x_j)$ is a kernel-weighted in-degree: it accumulates the votes received by x_j along its incoming KNN edges. For $h(u) = 1/u^2$, Eq. (22) reduces algebraically to the historical edge vote,

$$g_{i,\mathrm{inv_r2}}(x_j) = B_K(x_i)\frac{(R_K(x_i)/r_{ij})^2}{\sum_{x_k\in\mathcal{N}_K^+(x_i)}(R_K(x_i)/r_{ik})^2} = \frac{1}{r_{ij}^2} = \frac{1}{D_{ij}}. \tag{24}$$

The implementation uses this final expression directly for the inverse-square branch, avoiding a normalise–denormalise round-off cycle. Exact duplicate observations are excluded because the shared historical budget is undefined at $r = 0$.

5.3 RMS visibility in squared-distance units

The base, or unmodulated, visibility threshold depends only on the outgoing KNN distances of x_i and is the RMS of the stored squared distances,

$$V_{K,0}^{(2)}(x_i) = \left(\frac{1}{K}\sum_{x_j\in\mathcal{N}_K^+(x_i)} D_{ij}^2\right)^{1/2}. \tag{25}$$

Here, the subscript 0 denotes the absence of popularity modulation, rather than an additional vote model, and the superscript (2) indicates squared-distance units. Popularity then produces the effective threshold

$$V_{K,g}^{(2)}(x_i) = V_{K,0}^{(2)}(x_i)\left(1 + \frac{P_{K,g}(x_i)R_K^{(2)}(x_i)}{N}\right). \tag{26}$$

The subscript g records that this effective threshold uses the popularity computed under vote model g. Since $P_{K,g}(x_i) \geq 0$, modulation cannot reduce the base threshold. Equations (24)–(26) reproduce the historical inverse-square implementation exactly. The comparison models change only the within-source allocation in Eq. (22). Under a global coordinate rescaling $r \mapsto cr$, all four vote families scale as c^{-2} because $B_K \mapsto c^{-2}B_K$, while $R_K^{(2)} \mapsto c^2 R_K^{(2)}$; hence the popularity factor $P_{K,g}R_K^{(2)}$ is invariant for every model. Feature-wise preprocessing can still change the KNN graph and must therefore be specified and reused exactly.

5.4 Visible forward and backward steps

The visible forward neighbourhood of x_i is

$$\mathcal{V}_K^+(x_i) = \{x_j \in \mathcal{N}_K^+(x_i) : D_{ij} \leq V_{K,g}^{(2)}(x_i)\}. \tag{27}$$

The visible backward neighbourhood of x_i is

$$\mathcal{V}_K^-(x_i) = \{x_\ell \in \mathcal{N}_K^-(x_i) : D_{\ell i} \leq V_{K,g}^{(2)}(x_\ell)\}. \tag{28}$$

Algorithm 1 Visibility-filtered round-trip closure for one point

Require: directed KNN graph $\mathcal{G}_K$, vote model g, visibility thresholds $V_{K,g}^{(2)}$, point x_i

$C \leftarrow \{x_i\}$
repeat
  $C_{old} \leftarrow C$
  for all $x_q \in C_{old}$ **do**
    $C \leftarrow C \cup \mathcal{V}_K^+(x_q)$
    $C \leftarrow C \cup \mathcal{V}_K^-(x_q)$
  end for
until $C = C_{old}$
if $|C| > 1$ **then**
  return C
else
  return rejected singleton closure
end if

The backward condition is evaluated from the viewpoint of the reverse neighbour x_ℓ: a reverse-neighbour step is accepted only if $D_{\ell i}$ does not exceed the squared-distance visibility threshold of x_ℓ. The one-step visibility-filtered round-trip neighbourhood is therefore

$$\mathcal{R}_K^{\mathrm{vis}}(x_i) = \mathcal{V}_K^+(x_i) \cup \mathcal{V}_K^-(x_i). \tag{29}$$

5.5 Recursive closure and singleton rejection

Starting from a seed point x_i, the recursive closure is

$$C_K^0(x_i) = \{x_i\}, \tag{30}$$

$$C_K^{t+1}(x_i) = C_K^t(x_i) \cup \bigcup_{x_q \in C_K^t(x_i)} \mathcal{R}_K^{\mathrm{vis}}(x_q). \tag{31}$$

The process saturates when

$$C_K^{t+1}(x_i) = C_K^t(x_i). \tag{32}$$

The saturated closure is denoted

$$C_K^\infty(x_i) = C_K^s(x_i), \tag{33}$$

where s is the first saturation time.

A saturated closure is accepted as a cluster only when it contains more than one point:

$$|C_K^\infty(x_i)| > 1. \tag{34}$$

This singleton-rejection rule prevents isolated points or failed expansions from being reported as meaningful clusters. In the reported synthetic experiments, the accepted layers are subsequently checked to verify whether they cover the full dataset and therefore form proper partitions.

5.6 Optional structural pruning of saturated closures

In the unpruned method, one visible path is sufficient to place its endpoints in the same saturated closure. Structural pruning is an optional second stage, enabled in the native executable by `--structural-pruning`, that tests whether a large unpruned closure contains strongly supported internal populations. It is disabled by default, so the historical inverse-square regression and the four-kernel ablation remain unchanged. The native selector `--vote-kernel` accepts `uniform`, `inv_r`, `inv_r2`, or `sigmoid`; `inv_r2` remains the default, and the sigmoid slope is controlled by `--sigmoid-alpha` (default 8). The selected kernel supplies both the popularity-modulated visible graph and the popularity-ranked first mixture seed, while every subsequent pruning criterion is held fixed.

Let C be an unpruned saturated closure. It is considered for pruning only when

$$|C| \geq n_{\text{parent}} = \max\{50, \lceil 0.15N \rceil\}. \tag{35}$$

Each coordinate is standardised within C, and diagonal-covariance Gaussian mixtures with $m \in \{1, 2, 3, 4\}$ components are fitted by expectation–maximisation [27]. For dimension d, the implementation compares

$$\mathrm{BIC}_m = -2\ell_m + p_m \log |C|, \qquad p_m = m(2d + 1) - 1, \tag{36}$$

where ℓ_m is the maximised log likelihood and p_m counts mixture weights, means, and diagonal variances [28]. Models with $m > 1$ use five deterministic restarts. In the first restart, the initial centre is the point of greatest reverse-neighbour popularity and subsequent centres are chosen by farthest-first traversal; this is the operational role of leadership in the pruning stage. The remaining restarts use deterministic seeds, making repeated executions reproducible.

Let $m^* = \arg\min_m \mathrm{BIC}_m$. A candidate is retained only if

$$m^* \in \{2, 3\}, \qquad \mathrm{BIC}_1 - \mathrm{BIC}_{m^*} \geq 10, \tag{37}$$

and every proposed child contains at least

$$n_{\text{child}} = \max\{20, \lceil 0.05|C| \rceil\} \tag{38}$$

points. Thus, a four-component optimum is a conservative veto rather than an instruction to subdivide a curved or extensively fragmented closure.

Mixture membership alone does not determine the output. Each proposed child must also be coherent in the symmetrised visible KNN graph. A fragment smaller than n_{child} is reassigned to the alternative child with the greatest visible-edge support; the proposal is rejected if a child retains multiple large fragments or if a size constraint fails after regularisation. For a candidate labelling z_C that satisfies the mixture, size, and graph-coherence criteria, the crossing set is

$$E_C^{\text{cut}} = \left\{ \{i, j\} \in \overline{E}_{K,g}^{\text{vis}}[C] : z_C(i) \neq z_C(j) \right\}, \tag{39}$$

where $\overline{E}^{\mathrm{vis}}_{K,g}$ denotes the visible graph after disregarding edge orientation. The subdivision is accepted only when $|E^{\mathrm{cut}}_C| > 0$; its regularised children replace C in the final layer. The native implementation does not modify the KNN adjacency lists; instead, it rebuilds the cluster layer from the regularised child assignments. At the level of the resulting partition, this is equivalent to treating the crossing connections as cut. Reference labels play no role in proposal generation, model selection, regularisation, or final reassignment.

6 Cluster layers and multiscale evolution

For a fixed K, the accepted saturated closures define a cluster layer

$$\mathcal{P}_K = \{C^{(K)}_1, \ldots, C^{(K)}_{m_K}\}. \tag{40}$$

By varying K over a prescribed set of scales

$$K \in \{K_{\min}, K_{\min}+1, \ldots, K_{\max}\}, \tag{41}$$

one obtains a multiscale sequence

$$\mathcal{P}_{K_{\min}}, \mathcal{P}_{K_{\min}+1}, \ldots, \mathcal{P}_{K_{\max}}. \tag{42}$$

In the controlled experiments, each layer is an independent execution: $\mathcal{G}_K$, $R^{(2)}_K$, popularity, and visibility are constructed for that same value of K by invoking the supported interface with $K_0 = K$. This is an explicit experimental convention, not a claim that the intermediate layers of one raw $K_0 < K_{\max}$ process are numerically identical. In the latter mode, the neighbour list is allocated at $K_{\max}$ before the internal loop, so a stored construction-time radius may remain tied to $K_{\max}$ at its smaller layers. Independent execution makes every reported $\mathcal{P}_K$ a function of one stated neighbourhood scale and prevents this second scale from being implicit.

Increasing K tends to increase graph connectivity. Therefore, small clusters from lower-K layers are expected to be absorbed into larger clusters at higher K values, although the exact behaviour depends on the dataset and on the visibility rule.

6.1 Inter-layer overlap

The overlap between clusters in consecutive layers is

$$W^{(K,K')}_{ab} = |C^{(K)}_a \cap C^{(K')}_b|. \tag{43}$$

Useful normalisations include

$$F^{(K,K')}_{ab} = \frac{|C^{(K)}_a \cap C^{(K')}_b|}{|C^{(K)}_a|}, \tag{44}$$

and the Jaccard index

$$J_{ab}^{(K,K')} = \frac{|C_a^{(K)} \cap C_b^{(K')}|}{|C_a^{(K)} \cup C_b^{(K')}|}. \tag{45}$$

Inter-layer overlap is expected and informative: it describes how clusters from one scale are preserved, absorbed, or merged at another scale. This is distinct from intra-layer overlap, which would indicate that a given layer is not a proper partition.

6.2 Saturation scale

A beneficial side effect of the K-layer construction is that the neighbourhood scale can be selected from the saturation behaviour of the induced cluster-layer sequence. We denote by K_{sat} the first value of K for which the accepted layer becomes stable under further increases of the neighbourhood size, up to relabelling of cluster indices. Lower values of K often expose local fluctuations, microfragmentation, or noise-like components, whereas the saturated layer represents the dominant macro-scale aggregation structure.

Operationally, saturation can be detected by monitoring the overlap between consecutive layers. If each cluster in $\mathcal{P}_K$ maps entirely to a unique cluster in $\mathcal{P}_{K+1}$, and vice versa, the two layers are equivalent. More robustly, one may require the same equivalence to persist over a short stability window,

$$\mathcal{P}_K \equiv \mathcal{P}_{K+1} \equiv \cdots \equiv \mathcal{P}_{K+h}, \tag{46}$$

where h is a prescribed persistence parameter. The saturation scale can then be defined as

$$K_{\mathrm{sat}} = \min\left\{K : \mathcal{P}_K \equiv \mathcal{P}_{K+1} \equiv \cdots \equiv \mathcal{P}_{K+h}\right\}. \tag{47}$$

Here, $\equiv$ denotes equality of cluster layers up to relabelling of cluster indices.

7 Implementation agreement and partition diagnostic

An important consistency check is whether each accepted layer $\mathcal{P}_K$ is a true partition of the dataset. For each K, this can be verified by comparing the sum of accepted cluster cardinalities with the cardinality of the union of all accepted cluster members:

$$\sum_{a=1}^{m_K} |C_a^{(K)}| = \left|\bigcup_{a=1}^{m_K} C_a^{(K)}\right|. \tag{48}$$

When the equality in Eq. (48) holds and the union contains all N data points, the accepted layer is not merely a cover of the dataset, but a proper partition:

$$\bigcup_{a=1}^{m_K} C_a^{(K)} = X, \qquad C_a^{(K)} \cap C_b^{(K)} = \varnothing \quad \text{for } a \neq b. \tag{49}$$

Table 1 Unpruned inverse-square regression results for the variable-density blobs dataset. Each K was run independently with $K_0 = K$.

K	clusters	ARI	NMI	unclustered
2	595	0.0116	0.3991	0
3	277	0.0403	0.4491	0
4	99	0.2809	0.5716	0
5	18	0.6661	0.8017	0
6	6	0.7774	0.8941	0
7	5	0.7805	0.8981	0
8	4	0.7817	0.9035	0

This property is important because the multiscale object produced by the algorithm is then a sequence of valid partitions, rather than a sequence of overlapping covers. Inter-layer overlap is still expected and informative, but intra-layer overlap would indicate an invalid layer representation. If singleton closures are rejected, the diagnostic also verifies whether any points were left outside the accepted layer.

If the left-hand side of Eq. (48) is larger than the right-hand side, at least one point belongs to more than one cluster in the same layer.

Before comparing vote kernels, the additive Python implementation was tested exclusively with the unpruned historical inverse-square model. The C executable and Python implementation received the same feature columns, datasets, and isolated values $K = 2, \ldots, 16$. Pointwise partitions were compared independently of cluster-number permutations, including an exact comparison of the unclustered masks. All 120 comparisons (eight datasets times 15 values of K) agreed exactly. This is a software regression gate: it establishes that the Python experiment reproduces the selected C baseline, but it is not by itself evidence that either implementation recovers an external reference partition or every earlier execution convention. The vote-kernel benchmark was permitted to proceed only after this gate passed. After native kernel selection was introduced, a second gate compared the unpruned C and Python partitions for the budget-matched shapes $h(u) = 1/u$, $h(u) = 1/u^2$, and $h(u) = S_\alpha(u)$ on the variable-density blobs at $K = 8$ and the bridge at $K = 6$. All six pointwise comparisons also agreed exactly before the pruned sensitivity figures were produced.

Table 1 reports a representative subset for the variable-density blobs dataset. Since all $N = 2000$ points are represented exactly once and the unclustered count is zero, these layers are proper partitions. ARI and NMI compare each partition with the known reference labels and are invariant under a permutation of cluster identifiers.

This diagnostic is different from inter-layer overlap. Inter-layer overlap is expected and informative; it describes how clusters from one scale are preserved, split, or absorbed at another scale. Figure 3 connects the numerical diagnostic to the actual K-layer transitions. In the displayed $K = 5, 6, 7, 8$ sequence, every lower-K cluster is contained completely in exactly one cluster at the next scale: the overlap audit finds zero splits, so this particular sequence admits a merge-only transition forest. This is an observed property of this dataset and range, not an assumption that every RTKNNC layer sequence must be nested.

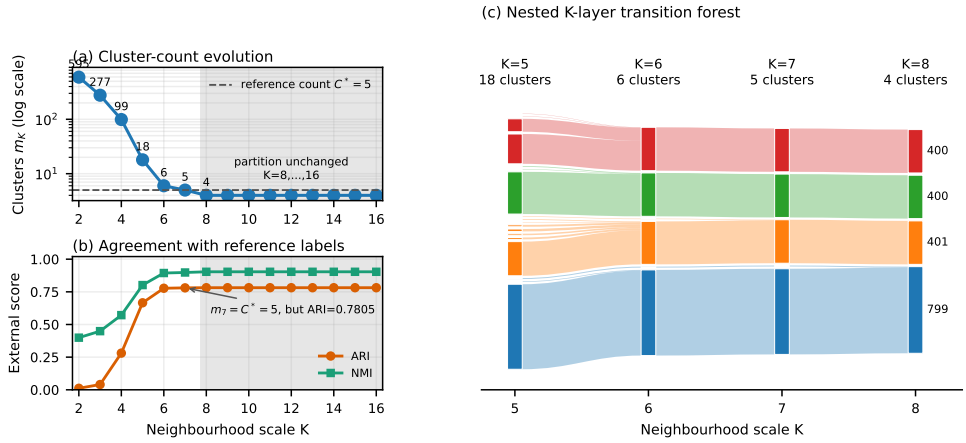


Fig. 3 Multiscale evolution of the unpruned historical inverse-square model on the variable-density blobs. Every layer is generated independently with $K_0 = K$. (a) The cluster count decreases from 595 microclusters at $K = 2$ to four components at $K = 8$ and is plotted on a logarithmic scale; the dashed line is the reference count $C^* = 5$, used only for evaluation. (b) ARI and NMI against the withheld reference labels. The shaded interval marks the exactly unchanged partition from $K = 8$ through the largest tested value, $K = 16$. (c) The merge-only transition forest over $K = 5, 6, 7, 8$. Node and ribbon thicknesses encode point population and overlap count $W_{ab}^{(K,K')}$, respectively. Colours identify the four lineages ending at $K = 8$ and do not encode reference labels. The equality $m_7 = C^*$ without high ARI and the stable $m_K = 4$ plateau demonstrate that cluster-count or partition stability alone does not guarantee recovery of latent classes.

The same pointwise audit shows that the unpruned partition at $K = 8$ remains unchanged through $K = 16$. Nevertheless, internal stability is not equivalent to external correctness. At $K = 7$, the cluster count already equals the five reference classes, but ARI is only 0.7805; at the tested-range saturation beginning at $K = 8$, four components remain and ARI is 0.7817. The structural pruning experiment in Sect. 11 addresses this specific stable merger without using the reference labels at execution time.

8 Scale, popularity, visibility, and saturation

The formulation suggests an interpretation in terms of propagation, popularity, and selectivity. In the present graph setting, this balance is implemented through the neighbourhood scale K, the kernel-dependent reverse-neighbour popularity score $P_{K,g}(x_i)$, and the local visibility threshold $V_{K,g}^{(2)}(x_i)$. Small values of K preserve local selectivity but may fragment the graph. Larger values of K increase propagation, while popularity-modulated RMS visibility prevents all KNN links from being treated as equally admissible.

The base RMS visibility threshold in Eq. (25) acts as a pointwise scale estimate in squared-distance units, while Eq. (26) introduces a popularity-dependent enlargement. In dense regions the base threshold is small and expansion remains selective.

Points receiving many influential incoming KNN links obtain higher popularity and may propagate through a larger visible neighbourhood. The kernel determines what constitutes an influential incoming link without changing the closure operator.

The singleton-rejection rule in Eq. (34) adds a second selectivity mechanism. A seed that cannot produce a nontrivial saturated closure is not reported as a cluster. Thus, the reported layer is determined not only by reachability in the directed KNN graph, but also by reverse-neighbour popularity, visibility-filtered propagation, and nontrivial closure size.

As K increases, the induced layers may pass from microfragmented local structures to stable macro-scale agglomerations. The saturation scale K_{sat} provides an empirical way to select the neighbourhood scale from the layer sequence itself, rather than fixing K solely as an external hyperparameter.

9 Experimental protocol

The controlled benchmark reuses the eight synthetic datasets already distributed with the implementation:

- Gaussian blobs with equal and variable densities;
- two moons and concentric circles;
- anisotropic Gaussian clusters;
- a uniform point cloud;
- two dense clusters connected by a sparse bridge;
- a Swiss-roll embedding.

The label column is used only for evaluation and is never included among the clustering features. The Swiss-roll labels are an artificial discretisation of a continuous manifold and its external scores are therefore interpreted descriptively. ARI and NMI are not reported for the uniform dataset because it has no nontrivial reference partition.

For every dataset, K ranges from 2 through 16. Each K is run independently with $K_0 = K$. The KNN graph is constructed once for a dataset–K pair and is reused unchanged by all four vote kernels. All models share the inverse-square source budget in Eq. (21) and therefore require strictly positive neighbour distances, so the input is checked for exact duplicate points before execution. Uniform, inverse-distance, inverse-square, and sigmoid refer to the four allocation shapes in Eq. (19); the sigmoid uses $\alpha = 8$. RMS visibility and singleton rejection are enabled for every model. Structural pruning is disabled throughout this 840-row comparison, preserving a controlled ablation in which only the within-source allocation changes.

The bridge benchmark varies the retained fraction of the existing labelled bridge points over $\{0, 0.10, 0.25, 0.50, 0.75, 1\}$. Core points are never subsampled. The two dense cores are considered merged when their dominant predicted cluster identifier is the same; this criterion is independent of the numerical names assigned to clusters.

The output is one long-format CSV. For each dataset, K, and vote model, it records:

- the dataset name, K, vote model, and complete model parameters;
- the number of clusters and unclustered points;

- ARI and NMI when a nontrivial reference partition is available [29, 30];
- the mean number of visible neighbours and mean visibility in squared-distance units;
- end-to-end runtime, including KNN-graph construction;
- the bridge-merger indicator for the parameterised bridge rows.

The same measured graph-construction time is included in each of the four model runtimes for a given dataset–K pair. This preserves end-to-end interpretation while ensuring that differences in graph construction cannot favour one vote kernel.

Structural pruning is evaluated separately in the native C executable under the historical inverse-square vote. Each case is run twice with identical coordinates and isolated K: once with the default unpruned closure and once with `--structural-pruning`. The selected cases are variable-density blobs at $K = 8$, the complete sparse bridge at $K \in \{6, 10\}$, and three preservation controls—circles at $K = 7$, two moons at $K = 10$, and anisotropic blobs at $K = 8$. The fixed implementation constants are one through four diagonal mixture components, five restarts for $m > 1$, a minimum BIC gain of 10, the parent and child sizes in Eqs. (35) and (38), and a four-component veto. The executable is passed `--dim 2`; the label column is read only afterwards to compute ARI and NMI. Complete native logs, layer files, input hashes, and a machine-readable parameter manifest accompany the figures.

To test whether the two principal corrections depend on the historical allocation shape, the variable-density blobs at $K = 8$ and the complete bridge at $K = 6$ are also run natively with the three budget-matched shapes $1/u$, $1/u^2$, and $S_\alpha(u)$, with $\alpha = 8$. Uniform allocation remains in the broader unpruned ablation but is excluded from this focused weighted-kernel check. For each kernel, the unpruned C result must first match the corresponding Python partition pointwise; the same native mixture, BIC, size, coherence, and pruning parameters are then applied without retuning.

9.1 External-method comparison protocol

The final comparison uses the six datasets having nontrivial discrete references and seven external methods: agglomerative clustering, k-means, Gaussian mixtures, DBSCAN, OPTICS, KNN-affinity spectral clustering, and HDBSCAN. The uniform cloud has no reference partition, and the Swiss-roll bands are an artificial discretisation of a continuous manifold; both are therefore excluded from this table. No method-specific standardisation is applied: every method receives the same raw feature columns.

Structurally pruned RTKNNC uses the historical inverse-square vote and sweeps $K = 2, \ldots, 16$ in the native executable. Agglomerative clustering evaluates Ward, complete, average, and single linkage, both without a connectivity constraint and with symmetric KNN connectivity at 5, 10, and 15 neighbours. k-means uses 20 initialisations; Gaussian mixtures use five initialisations under full, tied, diagonal, and spherical covariance models. These three methods and spectral clustering receive the reference cluster count C^*. Spectral clustering uses KNN affinity with $k_g = 2, \ldots, 16$, ARPACK eigenvectors, and 20 label-assignment restarts at random seed zero.

DBSCAN sweeps $m \in \{3, 5, 10, 20\}$ for `min_samples`. For each m, let $d_i^{(m)}$ be the distance from x_i to its $(m-1)$st proper neighbour, because DBSCAN counts

the query point itself. Its ϵ candidates are the 400 empirical quantiles of $\{d_i^{(m)}\}_{i=1}^N$ at $q = 0.0025, 0.005, \ldots, 1$. OPTICS combines the same four values of m with $\xi \in \{0.01, 0.025, 0.05, 0.1\}$ and minimum-cluster fractions $\{0.01, 0.025, 0.05\}$. HDBSCAN combines minimum cluster sizes $\{5, 10, 20, 50, 100\}$, `min_samples` in $\{\texttt{None}, 5, 10, 20\}$, and excess-of-mass or leaf selection. These three density-based methods infer their reported cluster counts.

For each method–dataset pair, the table reports the candidate with maximum ARI, then maximum NMI. Remaining external-method ties prefer fewer noise points and then the first candidate in the declared grid; RTKNNC ties prefer smaller K. Thus, the result is explicitly a label-selected oracle envelope measuring what each fixed grid can represent, not a claim that ARI supplies an unsupervised deployment rule. A noise identifier -1 participates in ARI and NMI as a predicted label, while the reported cluster count excludes it. The comparison comprises 10,344 external fits and 90 RTKNNC fits. Complete external sweeps, selected rows, raw selected labels, software versions, input hashes, and a provenance-checked table manifest accompany the article artifacts.

10 Vote-model benchmark results

The complete sweep contains 840 observations: 480 rows from eight datasets, 15 values of K, and four kernels, plus 360 rows from six bridge fractions, 15 values of K, and four kernels. No observation contained an unclustered point. Among the 210 matched dataset–scale configurations, the number of clusters differed across kernels in 60 configurations; ARI and NMI differed in 57 labelled configurations. The mean number of visible neighbours and the mean visibility both differed in all 210 configurations. Thus, the allocation shape frequently changes the internal visibility dynamics, although the resulting external optimum can remain stable.

Figure 4 displays the complete cluster-count trajectory for each primary dataset. Every case begins with substantial microfragmentation and then consolidates rapidly, but the macro-scale behaviour is dataset dependent. Some labelled datasets reach the reference count only over a finite interval of K, whereas others stabilise at it or merge below it. This complements the external scores: agreement in cluster count identifies a candidate scale, but does not establish agreement with the reference partition.

For anisotropic clusters, equal-density blobs, circles, and moons, every kernel attained ARI=NMI=1 within the tested K range. In this unpruned ablation, all kernels also attained the same best result for variable-density blobs (ARI 0.7817, NMI 0.9035, $K = 8$) and for the labelled bridge dataset (ARI 0.8083, NMI 0.8182, $K = 6$). These are unpruned optima, not the final results of the structural-pruning enhancement reported in Sect. 11. For the discretised Swiss roll, the best scores were low, as expected for an artificial banding of a continuous manifold. All optima occurred at $K = 4$: uniform, inverse-distance, inverse-square, and sigmoid allocation attained ARI/NMI 0.1559/0.5160, 0.1544/0.5141, 0.1528/0.5131, and 0.1558/0.5153, respectively.

For the coloured comparison figures only, a common representative K is selected for each labelled dataset by maximising the mean ARI across the four kernels, with

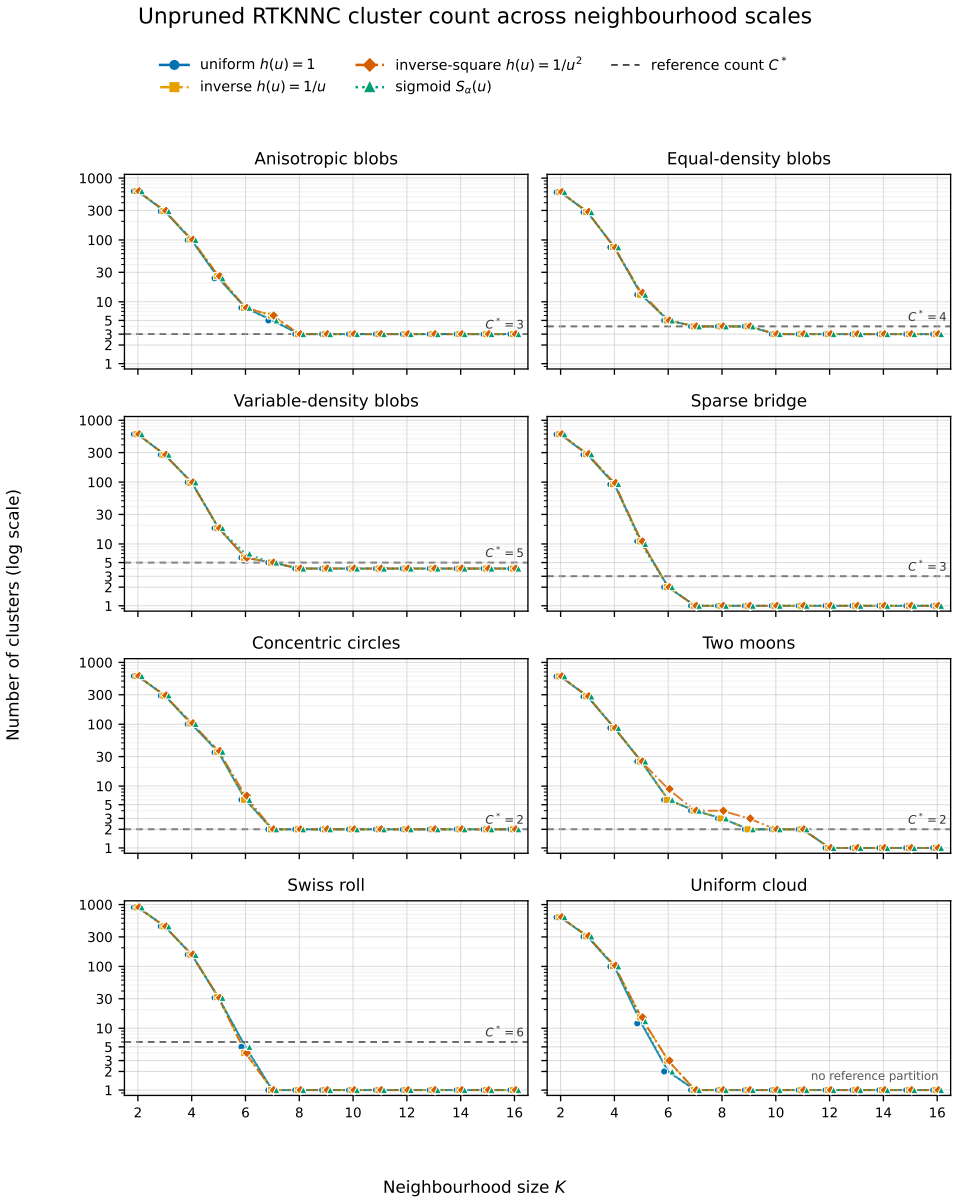


Fig. 4 Number of clusters as a function of K for every primary dataset in the controlled, unpruned four-kernel benchmark. Each point is obtained from an independent execution with $K_0 = K$, and the ordinate is logarithmic. Lines are evaluated at the integer K values; markers have small horizontal offsets only to expose coincident counts from different kernels. Dashed lines show the reference count C^* for evaluation only. The Swiss-roll reference comprises artificial bands, and the uniform cloud has no reference partition. Matching C^* is therefore neither an execution-time selection rule nor evidence of pointwise partition correctness.

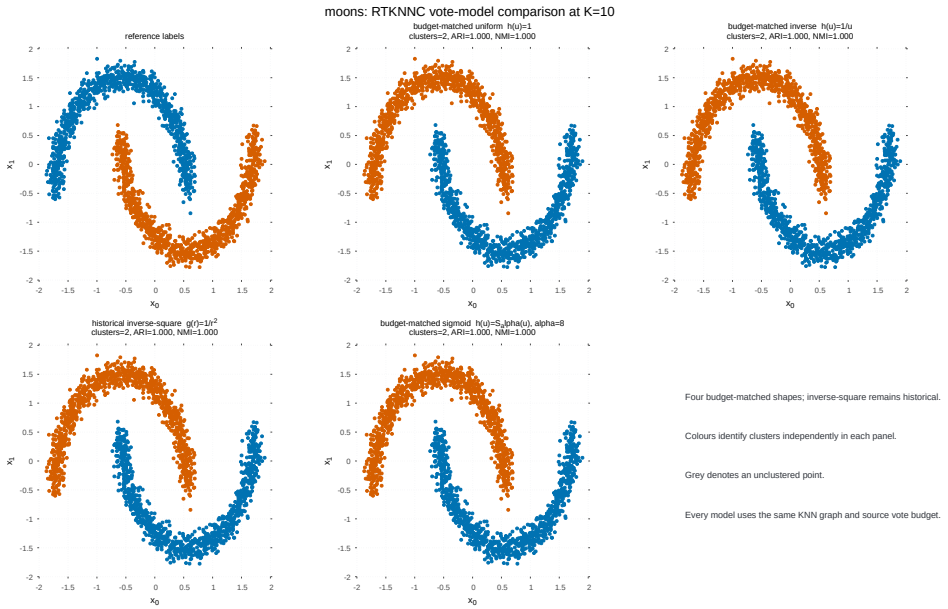


Fig. 5 Reference labels and the four predicted partitions for the two-moons dataset at the common representative scale $K = 10$: budget-matched uniform $h(u) = 1$, inverse-distance $h(u) = 1/u$, historical inverse-square $h(u) = 1/u^2$, and local sigmoid $S_\alpha(u)$ with $\alpha = 8$. Every source casts the common budget in Eq. (21); the panels differ only in its allocation among outgoing neighbours. Every vote model recovers the two non-convex components exactly (ARI = NMI = 1). The GNUplot rendering explicitly labels both coordinate axes; cluster colours are local to each panel and do not encode a cross-panel label correspondence.

ties resolved in favour of the smaller K. The same selected K is then used in every predicted panel, so the graphical comparison does not grant a different scale to each kernel. This visualisation rule does not filter the long-format results, which retain all $K = 2, \ldots, 16$ observations.

Figure 5 provides a representative non-convex case in which all four kernels recover the reference partition exactly. The more difficult variable-density case is reserved for the focused, pruned three-kernel mosaic in Fig. 7.

The cluster count alone can conceal this failure. At $K = 6$, the six reported components do not include the compact overlapping Gaussian as an independent cluster: one component contains all 400 of its points together with 390 of the 400 points from the diffuse Gaussian. The two additional components are small fragments. At $K = 8$, the merged component contains all 400 compact points and 399 diffuse points. No tested value $K = 2, \ldots, 16$ in the *unpruned* four-kernel sweep recovers the five-class reference partition: smaller K fragments the point cloud before the overlapping distributions separate, whereas larger K connects them. The failure arises in the connectivity closure and is corrected by the native structural refinement in Fig. 7, whose execution does not use the reference labels.

Table 2 First K producing a merger of the two dense bridge cores for each of the four vote kernels. A value > 16 means that no merger occurred in the tested range.

bridge fraction	$h(u) = 1$	$h(u) = 1/u$	$h(u) = 1/u^2$	$S_\alpha(u)$
0.00	> 16	> 16	> 16	> 16
0.10	5	5	6	5
0.25	6	6	6	6
0.50	6	6	6	6
0.75	7	7	7	7
1.00	7	7	7	7

10.1 Bridge-fraction transition

Table 2 reports the first value of K at which the two dense cores share a dominant predicted cluster. No kernel merged the disconnected cores when all bridge points were removed. At bridge fraction 0.10, the historical inverse-square allocation merges one scale later than the other three; the boundaries coincide at the remaining tested fractions. This is direct evidence that the budget-matched shapes are not equivalent implementations.

For the complete bridge at $K = 6$, the two reported components also do not correspond to the three reference labels. One contains all 900 points of the left core, all 200 bridge points, and six peripheral points from the right core; the other contains the remaining 894 right-core points. The bridge has therefore already been absorbed by the left component even though the two dense cores have not yet fully merged.

11 Native structural-pruning results

Table 3 separates the native pruning experiment from the four-kernel sweep. The variable-density case changes from four to five clusters at $K = 8$, with ARI increasing from 0.7817 to 0.9627. The accepted subdivision has 12 crossing visible connections. The result is not a relabelling of the reference partition: the five inferred cluster sizes are 375, 400, 400, 401, and 424, so the remaining disagreement is reported rather than hidden by the correct cluster count.

For the complete sparse bridge, pruning recovers three components at both tested scales. At $K = 6$, ARI increases from 0.8083 to 0.9853, and 10 visible connections cross the accepted subdivision. At $K = 10$, the unpruned closure has already collapsed to one component, but an accepted three-child subdivision has 32 crossing connections and reaches ARI 0.9865. Circles, moons, and anisotropic blobs serve as preservation controls: no split is accepted, and their ARI remains exactly 1.

11.1 Pruning sensitivity to the vote kernel

Figures 6 and 7 address whether the successful split is an artefact of the historical inverse-square vote. For the variable-density blobs, all three weighted allocation shapes start from four unpruned components and produce the same five-cluster partition after pruning: ARI 0.9627, NMI 0.9614. Each accepted subdivision has 12 crossing visible connections. The three partitions are pointwise identical up to label permutation.

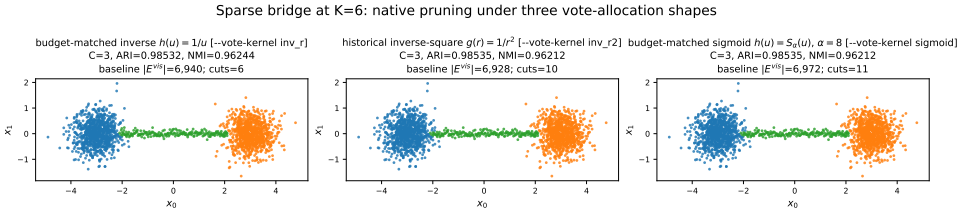


Fig. 6 Native structural pruning of the complete sparse bridge at $K = 6$ under the three weighted allocation shapes. The panels show budget-matched $h(u) = 1/u$, historical $h(u) = 1/u^2$, and budget-matched local sigmoid $S_\alpha(u)$ with $\alpha = 8$, respectively. Every model produces three clusters. Inverse-square and sigmoid allocation coincide pointwise (ARI 0.98535, NMI 0.96212), while inverse-distance differs from them at four points after label alignment (ARI 0.98532, NMI 0.96244). The accepted subdivisions have 6, 10, and 11 crossing visible connections, and their annotated native unpruned visible-edge counts are 6940, 6928, and 6972, respectively. Each panel comes from a separate native execution whose recorded command contains the displayed `--vote-kernel` token; the equal second and third scores reflect a pointwise-identical final partition despite distinct unpruned graphs, not reuse of a layer file. The reference panel is intentionally omitted; reference labels are not available to the clustering or pruning stages.

Table 3 Native inverse-square results before and after structural pruning. C^* is the number of reference classes, C_0 and C_p are the unpruned and pruned cluster counts, and $|E^{\mathrm{cut}}|$ is the number of crossing visible connections in the accepted subdivision (zero when no split is accepted). Labels are used only for the reported ARI and C^*.

dataset	K	C^*	C_0	ARI_0	C_p	ARI_p	$\lvert E^{\mathrm{cut}}\rvert$
Variable-density blobs	8	5	4	0.7817	5	0.9627	12
Sparse bridge	6	3	2	0.8083	3	0.9853	10
Sparse bridge	10	3	1	0.0000	3	0.9865	32
Circles	7	2	2	1.0000	2	1.0000	0
Two moons	10	2	2	1.0000	2	1.0000	0
Anisotropic blobs	8	3	3	1.0000	3	1.0000	0

Their mean visible-neighbour counts are not identical (4.7010, 4.6825, and 4.7350, respectively), confirming that the kernels alter the visible graph. In the companion Python edge audit, whose partitions pass the native agreement gate, the inverse-square directed edge set differs from the inverse-distance and sigmoid sets by 89 and 269 visible edges, respectively. These changes remain inside the same connected components, and the accepted pruning split therefore coincides at this scale.

The bridge result is also stable in its scientific conclusion. Every kernel starts from two components and returns the intended three, with ARI between 0.98532 and 0.98535. Inverse-square and sigmoid allocation induce the same partition, while inverse-distance differs by four peripheral assignments after optimal label alignment. Their accepted subdivisions have 10, 11, and 6 crossing connections, respectively, when listed as inverse-square, sigmoid, and inverse-distance. In the companion Python edge audit, the inverse-square unpruned visible edge set differs by 56 and 190 directed edges from inverse-distance and sigmoid voting, respectively. The equality of cluster counts

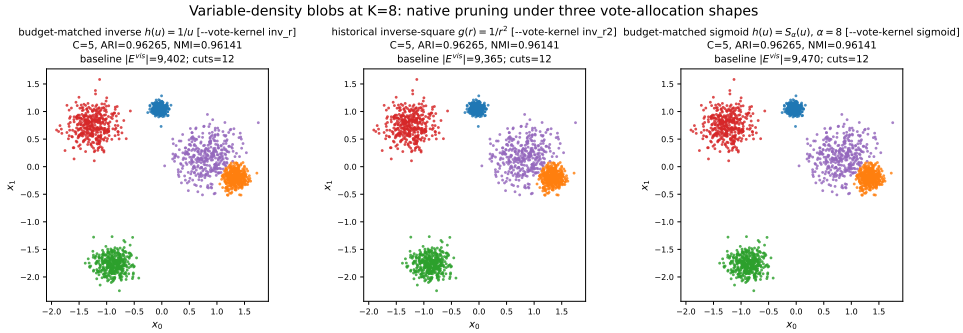


Fig. 7 Native structural pruning of the variable-density blobs at $K = 8$ under the three weighted allocation shapes. The three panels show, from left to right, budget-matched $h(u) = 1/u$, historical $h(u) = 1/u^2$, and budget-matched local sigmoid $S_\alpha(u)$ with $\alpha = 8$. All produce the same pointwise five-cluster partition (ARI 0.96265, NMI 0.96141) after structural refinement; each accepted subdivision has 12 crossing visible connections. The panel annotations expose the distinct native unpruned graphs: $|E^{\mathrm{vis}}| = 9402$, 9365, and 9470 directed edges, respectively. Reference labels are omitted from the mosaic and are used only for colour alignment and external scores; each unpruned C partition was first verified pointwise against its Python kernel definition.

is therefore supported by near-identical external partitions, not used as a substitute for them.

12 Comparison with external clustering methods

Table 4 reports the oracle envelope defined in Sect. 9. RTKNNC attains ARI = NMI = 1 on anisotropic blobs, equal-density blobs, circles, and two moons. Several external methods tie these results, but the controls also expose their inductive biases: k-means and Gaussian mixtures fail on circles and two moons, whereas graph-, density-, and connectivity-constrained methods recover the non-convex partitions. In particular, the degradation of k-means is expected rather than an implementation anomaly. Its nearest-centroid assignment induces convex Voronoi cells, which cannot represent two concentric rings adequately and only approximates the curved, interleaving moons. Consequently, it obtains ARI/NMI $-0.0004/0.0001$ on circles and $0.4800/0.3815$ on two moons. This limitation persists even under the favourable protocol used here, in which k-means is given the reference cluster count $C^* = 2$.

The two connectivity-stress cases provide finer discrimination. On the variable-density blobs, RTKNNC returns the intended five clusters with ARI 0.9627 and NMI 0.9614, narrowly below the full-covariance Gaussian-mixture result $0.9639/0.9636$. Spectral clustering reaches $0.9569/0.9581$, while agglomerative clustering, k-means, OPTICS, and HDBSCAN obtain ARIs between 0.9052 and 0.9306. DBSCAN reaches ARI 0.7916 and NMI 0.8044; its score-maximising candidate fragments the data into 15 clusters and marks 204 points as noise.

On the sparse bridge, RTKNNC attains the highest score, $0.9865/0.9632$, followed closely by the full-covariance Gaussian mixture at $0.9831/0.9552$. Spectral clustering reaches $0.9487/0.8993$, and the other external ARIs range from 0.6723 to 0.9342. The RTKNNC bridge envelope selects $K = 11$ by a marginal ARI advantage over $K = 10$;

Table 4 Best label-selected grid result for structurally pruned RTKNNC and external clustering methods. Each cell reports ARI/NMI; bold denotes the highest ARI in the row, including ties. Methods marked † receive the reference cluster count C^*; the others do not. Reference labels are used only for scoring and oracle selection, never as clustering features.

dataset	C^*	RTKNNC	Agglom.†	K-means†	GMM†	DBSCAN	OPTICS	Spectral†	HDBSCAN
Anisotropic	3	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**
Equal-density blobs	4	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	0.9987/0.9977	**1.0000/1.0000**	0.9940/0.9902
Variable-density blobs	5	0.9627/0.9614	0.9306/0.9444	0.9189/0.9338	**0.9639/0.9636**	0.7916/0.8044	0.9266/0.9262	0.9569/0.9581	0.9052/0.9075
Sparse bridge	3	**0.9865/0.9632**	0.9153/0.8580	0.9342/0.8780	0.9831/0.9552	0.8140/0.7422	0.6723/0.6491	0.9487/0.8993	0.8641/0.7828
Circles	2	**1.0000/1.0000**	**1.0000/1.0000**	-0.0004/0.0001	-0.0004/0.0001	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**	**1.0000/1.0000**
Two moons	2	**1.0000/1.0000**	**1.0000/1.0000**	0.4800/0.3815	0.5096/0.4080	**1.0000/1.0000**	0.9990/0.9972	**1.0000/1.0000**	**1.0000/1.0000**

the focused $K = 6$ experiment remains useful because it isolates the earliest corrected three-component partition under the three vote kernels.

The comparison therefore supports competitiveness rather than universal superiority. RTKNNC ties the best result on the standard controls, is within 0.0013 ARI of the best variable-density result, and leads on the sparse bridge. The fixed-count baselines have the advantage of receiving C^*, whereas RTKNNC and the three density-based methods do not. Repeated stochastic realisations, higher-dimensional data, and a label-free parameter-selection study remain necessary.

13 Discussion

RTKNNC combines established KNN-graph ingredients in a specific operator; it does not claim the first use of KNN graphs or replace all density-based methods. Its defining step is recursive forward and reverse-neighbour traversal under a popularity-modulated RMS visibility filter, followed by singleton rejection and saturation. The optional, leadership-seeded structural stage operates after closure and does not replace the graph operator.

Unpruned RTKNNC is defined by an algorithmic graph construction rather than by minimising an explicit objective function. Changing only the vote allocation altered the cluster count in 60 of 210 matched configurations and visibility in all 210, although several datasets had the same best external partition under all kernels. Vote allocation controls incoming evidence; the round-trip graph and its closure control how visible evidence propagates. We report structural pruning separately because its BIC criterion adds a local statistical model.

The inverse-square kernel retains a special implementation-level status: it is the historical rule and is preserved exactly. A raw comparison against 1, $1/r$, and a bounded sigmoid would, however, mix kernel shape with incompatible units and outgoing amplitudes; on these data, the raw sigmoid was consequently almost indistinguishable from uniform voting. The source-budget-matched family removes that confound by conserving $B_K(x_i)$ for every source and changing only how that mass is distributed among its K outgoing edges. It also makes $P_{K,g}R_K^{(2)}$ invariant under global coordinate rescaling for every model, not just inverse-square. Feature-wise transformations can still change the neighbour graph, so preprocessing remains part of the method specification.

On the two difficult datasets, the pruning result was not specific to inverse-square voting. This does not imply kernel equivalence. For the variable-density blobs, the three kernels produce pointwise-identical final partitions even though their native

mean visible-neighbour counts differ and the companion audit finds hundreds of kernel-dependent directed edges. Those edge changes occur within the same symmetrised connected components, and the mixture stage accepts the same geometric subdivision. For the bridge, the stability is only approximate: inverse-distance voting changes four assignments relative to the pointwise- identical inverse-square and sigmoid results, a distinction hidden by three-decimal ARI and NMI. The matching blob scores therefore identify an empirical topological stability plateau, while the bridge shows that small kernel effects remain measurable. These experiments establish robustness at the reported scales, not general kernel invariance.

The bridge-fraction experiment provides a local sensitivity check. At retained bridge fraction 0.10, uniform, inverse-distance, and sigmoid allocation first merge the dense cores at $K = 5$, whereas historical inverse-square first merges them at $K = 6$. This observed boundary difference confirms that equal results at the complete-bridge reference scale do not imply kernel equivalence. A denser sweep over bridge geometry, noise, separation, and sigmoid slope may expose further transition differences.

13.1 Connectivity failure and structural correction

The observed failures follow from the topology of the closure operator. For a fixed K and vote kernel g, define the visible directed edge set

$$E_{K,g}^{\mathrm{vis}} = \left\{ (i,j) : x_j \in \mathcal{N}_K^+(x_i),\ D_{ij} \le V_{K,g}^{(2)}(x_i) \right\}. \tag{50}$$

Forward expansion traverses outgoing edges in this graph and backward expansion traverses incoming edges. Consequently, apart from the explicit singleton policy, the *unpruned* saturated closures are the connected components obtained after disregarding the orientation of the visible edges. The unpruned method can therefore recover a reference partition exactly only at a scale for which its classes form separate components of this symmetrised visible graph. The enhanced method relaxes that restriction in a controlled way: Eqs. (37)–(39) permit a large component to be divided only when independent mixture and graph-coherence tests agree.

This condition explains why nonconvexity alone does not cause failure. Although moons and circles are nonconvex, at a suitable value of K the low-density regions between their sampled components leave no path connecting them in the symmetrised visible KNN graph. The variable-density example is different: the compact and diffuse right-hand Gaussians overlap in the observed space. In this dataset, points in the overlap create cross-class nearest-neighbour edges, and an admissible chain connects the two latent classes within the same unpruned saturated closure. Reducing K breaks some edges but produces microclusters before separating these classes; increasing K reinforces their connection. At $K = 8$, the structural stage detects two populations inside the merged closure: BIC favours a two-component subdivision, and both proposed children are coherent in the visible graph. This restores five clusters with ARI 0.9627. The remaining disagreement is consistent with the ambiguity created by overlapping coordinates, which do not define a uniquely recoverable boundary.

The sparse bridge creates the analogous phenomenon through a filament rather than direct overlap. Its left endpoint reaches vertices that already belong to the giant

left-core component, whereas the closest right-end vertices remain in small peripheral components until the next scale. Once a visible path reaches a giant component, recursive forward–backward expansion absorbs the entire path; a single cross-boundary connection is sufficient in the unpruned method. The native stage identifies three supported populations at $K = 6$; the accepted subdivision has 10 crossing connections and preserves the bridge as a third structure with ARI 0.9853. The Swiss roll presents the complementary case: its reference bands discretise one continuous manifold, so selective visibility yields many local components rather than the prescribed bands, while less selective scales eventually connect across them. The four-component veto and graph-coherence test are intended to protect such curved or fragmented structures, but a systematic manifold study remains necessary.

The unpruned output is therefore a partition of visible KNN connectivity, whereas the enhanced output is that partition after a conservative local model-based test of its large components. Neither is guaranteed to reconstruct every externally supplied label partition. The distinction matters scientifically: the four-kernel sweep measures vote sensitivity of the original closure, while the native comparison measures the incremental effect of structural pruning.

The present study has several limitations:

1. the synthetic results use one deterministic realisation of each dataset family;
2. the sigmoid experiment fixes $\alpha = 8$ rather than sweeping its slope, and the focused three-kernel pruning comparison is restricted to the two difficult reference cases;
3. the bridge benchmark varies bridge density but not width, curvature, core separation, or noise;
4. the pruning thresholds and diagonal-covariance family are fixed and have been evaluated here on six primary dataset–scale cases;
5. overlapping latent classes may remain only approximately identifiable, and broader tests are needed to quantify false splits on curved, heavy-tailed, and high-dimensional data;
6. the empirical runtime measurements are not a substitute for a formal complexity analysis;
7. the external comparison uses one implementation and one deterministic realisation of six two-dimensional datasets; it uses reference labels to select every grid optimum and supplies C^* to four fixed-count baselines, so label-free selection, repeated realisations, and comparisons with SNN and mutual-KNN methods remain open.

14 Conclusion

We presented a clustering method based on popularity-modulated, visibility-filtered round-trip closure in directed KNN graphs, separated its topological operator from four source-budget-matched vote-allocation shapes, preserved the historical inverse-square rule exactly, and added optional native structural pruning for overly connected closures. A 120-case inverse-square C–Python agreement test served as a software regression gate, and the unpruned 840-row sweep showed that the kernel often changes visibility and sometimes the induced partition. The native enhancement then corrected the central connectivity failures: variable-density blobs changed from four to

five clusters with ARI 0.7817 → 0.9627, and the bridge reached three clusters with ARI 0.9853 at $K = 6$ and 0.9865 at $K = 10$. It accepted no split on three already correct controls, whose ARI remained 1. At the two difficult reference scales, inverse-distance, inverse-square, and sigmoid voting all preserved the corrected cluster counts with essentially identical scores, and their six unpruned native partitions passed pointwise C–Python agreement. A 10,434-candidate oracle-envelope comparison with seven external methods then showed that pruned RTKNNC ties the perfect result on the standard controls, nearly matches the best variable-density result (0.9627 versus 0.9639 for a full-covariance Gaussian mixture), and attains the highest sparse-bridge ARI (0.9865). These results show that vote weighting, visible connectivity, and structural correction are distinct experimental axes. Broader replication is still required for the fixed pruning thresholds, high-dimensional data, alternative covariance models, label-free scale selection, and additional graph-based baselines.

Acknowledgements. To be completed.

Declarations.

- Funding: To be completed.
- Conflict of interest/Competing interests: The authors declare that they have no competing interests. To be confirmed.
- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Data availability: The synthetic datasets used in this study, including their reference labels, are publicly available in the project repository at https://github.com/epmarinho/rtknnc.
- Materials availability: Not applicable.
- Author contribution: To be completed.
- Use of generative AI and AI-assisted technologies: During preparation of this manuscript, the authors used OpenAI ChatGPT and Codex to assist with language editing, manuscript organisation, code review, and the development of reproducibility and figure-generation scripts. All AI-assisted material was reviewed and verified by the authors, who take full responsibility for the content.

References

[1] Cover, T.M., Hart, P.E.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory **13**(1), 21–27 (1967) https://doi.org/10.1109/TIT.1967.1053964

[2] Loftsgaarden, D.O., Quesenberry, C.P.: A nonparametric estimate of a multivariate density function. The Annals of Mathematical Statistics **36**(3), 1049–1051 (1965) https://doi.org/10.1214/aoms/1177700079

[3] Bentley, J.L.: Multidimensional binary search trees used for associative searching. Communications of the ACM **18**(9), 509–517 (1975) https://doi.org/10.1145/361002.361007

[4] Friedman, J.H., Bentley, J.L., Finkel, R.A.: An algorithm for finding best matches in logarithmic expected time. ACM Transactions on Mathematical Software **3**(3), 209–226 (1977) https://doi.org/10.1145/355744.355745

[5] Luxburg, U.: A tutorial on spectral clustering. Statistics and Computing **17**(4), 395–416 (2007) https://doi.org/10.1007/s11222-007-9033-z

[6] Jarvis, R.A., Patrick, E.A.: Clustering using a similarity measure based on shared near neighbors. IEEE Transactions on Computers **C-22**(11), 1025–1034 (1973) https://doi.org/10.1109/T-C.1973.223640

[7] Ertöz, L., Steinbach, M., Kumar, V.: Finding clusters of different sizes, shapes, and densities in noisy, high dimensional data. In: Proceedings of the 2003 SIAM International Conference on Data Mining, pp. 47–58. SIAM, Philadelphia, PA, USA (2003). https://doi.org/10.1137/1.9781611972733.5

[8] Brito, M.R., Chávez, E.L., Quiroz, A.J., Yukich, J.E.: Connectivity of the mutual k-nearest-neighbor graph in clustering and outlier detection. Statistics & Probability Letters **35**(1), 33–42 (1997) https://doi.org/10.1016/S0167-7152(96)00213-1

[9] Gowda, K.C., Krishna, G.: Agglomerative clustering using the concept of mutual nearest neighbourhood. Pattern Recognition **10**(2), 105–112 (1978) https://doi.org/10.1016/0031-3203(78)90018-3

[10] Chowdhury, S., Amorim, R.C.: An efficient density-based clustering algorithm using reverse nearest neighbour. In: Intelligent Computing: Proceedings of the 2019 Computing Conference, Volume 2. Advances in Intelligent Systems and Computing, vol. 998, pp. 29–42. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-22868-2_3

[11] Xie, W.-B., Lee, Y.-L., Wang, C., Chen, D.-B., Zhou, T.: Hierarchical clustering supported by reciprocal nearest neighbors. Information Sciences **527**, 279–292 (2020) https://doi.org/10.1016/j.ins.2020.04.016

[12] Marinho, E.P., Callegari Junior, N., Aparecido Breve, F., Mazzoni Ranieri, C.: Clustering astronomical orbital synthetic data using advanced feature extraction and dimensionality reduction techniques. Neural Computing and Applications **38**, 340 (2026) https://doi.org/10.1007/s00521-026-12057-4

[13] Liu, Y., Zhang, L., Xie, J., Wang, M., Yu, J.X., Cui, J.: Efficient Coreset Selection via K-Nearest Neighbor Graphs. Preprint, submitted 17 August 2026 (2026). https://arxiv.org/abs/2608.16270

[14] Sardana, D., Bhatnagar, R.: Graph clustering using mutual K-nearest neighbors. In: Active Media Technology. Lecture Notes in Computer Science, vol. 8610, pp. 35–48. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-09912-5_4

[15] Xie, J., Xiang, X., Xia, S., Jiang, L., Wang, G., Gao, X.: MGNR: A multi-granularity neighbor relationship and its application in KNN classification and clustering methods. IEEE Transactions on Pattern Analysis and Machine Intelligence **46**(12), 7956–7972 (2024) https://doi.org/10.1109/TPAMI.2024.3400281

[16] Ding, S., Li, M., Huang, T., Zhu, W.: Local density based on weighted K-nearest neighbors for density peaks clustering. Knowledge-Based Systems **305**, 112609 (2024) https://doi.org/10.1016/j.knosys.2024.112609

[17] Fei, Z., Ma, Y., Zhao, J., Wang, B., Yang, J.: KNEG-CL: Unveiling data patterns using a K-nearest neighbor evolutionary graph for efficient clustering. Information Sciences **690**, 121602 (2025) https://doi.org/10.1016/j.ins.2024.121602

[18] Ester, M., Kriegel, H.-P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining. KDD'96, pp. 226–231. AAAI Press, Menlo Park, CA, USA (1996)

[19] Ankerst, M., Breunig, M.M., Kriegel, H.-P., Sander, J.: Optics: Ordering points to identify the clustering structure. In: Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data, pp. 49–60. ACM, New York, NY, USA (1999). https://doi.org/10.1145/304181.304187

[20] Campello, R.J.G.B., Moulavi, D., Sander, J.: Density-based clustering based on hierarchical density estimates. In: Pei, J., Tseng, V.S., Cao, L., Motoda, H., Xu, G. (eds.) Advances in Knowledge Discovery and Data Mining. Lecture Notes in Computer Science, vol. 7819, pp. 160–172. Springer, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37456-2_14

[21] Campello, R.J.G.B., Moulavi, D., Zimek, A., Sander, J.: Hierarchical density estimates for data clustering, visualization, and outlier detection. ACM Transactions on Knowledge Discovery from Data **10**(1), 5–1551 (2015) https://doi.org/10.1145/2733381

[22] Ng, A.Y., Jordan, M.I., Weiss, Y.: On spectral clustering: Analysis and an algorithm. In: Advances in Neural Information Processing Systems, vol. 14, pp. 849–856 (2001)

[23] Murtagh, F., Contreras, P.: Algorithms for hierarchical clustering: An overview. WIREs Data Mining and Knowledge Discovery **2**(1), 86–97 (2012) https://doi.org/10.1002/widm.53

[24] Lloyd, S.P.: Least squares quantization in pcm. IEEE Transactions on Information Theory **28**(2), 129–137 (1982) https://doi.org/10.1109/TIT.1982.1056489

[25] MacQueen, J.: Some methods for classification and analysis of multivariate observations. In: Proceedings of the Fifth Berkeley Symposium on Mathematical

Statistics and Probability, vol. 1, pp. 281–297. University of California Press, Berkeley, CA (1967)

[26] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine learning in python. Journal of Machine Learning Research **12**, 2825–2830 (2011)

[27] Dempster, A.P., Laird, N.M., Rubin, D.B.: Maximum likelihood from incomplete data via the EM algorithm. Journal of the Royal Statistical Society: Series B (Methodological) **39**(1), 1–22 (1977) https://doi.org/10.1111/j.2517-6161.1977.tb01600.x

[28] Schwarz, G.: Estimating the dimension of a model. The Annals of Statistics **6**(2), 461–464 (1978) https://doi.org/10.1214/aos/1176344136

[29] Hubert, L., Arabie, P.: Comparing partitions. Journal of Classification **2**(1), 193–218 (1985) https://doi.org/10.1007/BF01908075

[30] Vinh, N.X., Epps, J., Bailey, J.: Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. Journal of Machine Learning Research **11**, 2837–2854 (2010)